HAVACILIKTA YAPAY ZEKA ARAŞTIRMALARI TOPLULUĞU

PROJE :	TUSAŞ Lift Up
GÖREV:	Kubik İnterpolasyon
TARİH:	20.03.2026

YÖNTEM

Algoritma Genel Akışı

Kübik Spline (Cubic Spline) interpolasyon algoritması, kesikli veri setlerini (örneğin uçuş hızları ve sürükleme katsayıları gibi aerodinamik ölçüm noktalarını) pürüzsüz ve sürekli bir eğriye dönüştürmek için ardışık ve sistematik bir matematiksel prosedür izler. Süreç; ham veri aralıklarının hesaplanıp sistemin temel iskeleti olan üç köşegenli matrisin (tridiagonal matrix) kurulmasıyla başlar. Ardından, eğrinin başlangıç ve bitiş davranışlarını fiziksel gerçekliğe sabitleyen sınır koşulları sisteme entegre edilir. Oluşan bu doğrusal denklem ağı, Thomas Algoritması kullanılarak yüksek bir bellek verimliliğiyle çözülür ve eğrinin tüm bükülme katsayıları elde edilir. Matematiksel iskeletin tamamlanmasının ardından algoritma, aranan özel bir hedef değerin veri setindeki tam konumunu İkiye Bölme (Bisection) yöntemiyle tespit eder ve nihai ara değer (interpolasyon) polinomunu hesaplayarak işlemi sonlandırır. Bu yapısal hiyerarşi, sistemin hem matematiksel olarak kararlı hem de donanımsal olarak optimize çalışmasını garanti eder.

Kübik Spline Algoritmasının İşlem Adımları

1. Veri Hazırlığı ve Adım Aralıklarının Hesaplanması

2. Üç Köşegenli Matrisin (Tridiagonal Matrix) Kurulması

3. Sınır Koşullarının (Boundary Conditions) Sisteme Uygulanması

4. Denklem Sisteminin Çözülmesi (Thomas Algoritması)

4.1. LU Ayrıştırma Aşaması (Decomposition Phase): Alt köşegenin sıfırlanması

4.2. Sayısal Çözüm Aşaması (Solution Phase): İleri ve geriye doğru yerine koyma ile k değerlerinin tamamen bulunması

5. Aranan Değerin Hangi Aralığa Düştüğünün Bulunması

6. Ara Değerin (İnterpolasyon Sonucunun) Hesaplanması

1. Veri Hazırlığı ve Adım Aralıklarının Hesaplanması

Kübik Spline algoritmasının ilk aşaması, sisteme girilen aerodinamik veri noktalarının (uçuş hızları ve sürükleme katsayıları gibi x ve y koordinatlarının) birbirleri arasındaki mesafelerin ve değişim oranlarının hesaplanmasıdır. Tüm veri düğümleri (knots) için ardışık x değerleri arasındaki yatay adım aralıkları ve bu aralıklara denk gelen y koordinatı değişimleri tespit edilir. **Kiusalaas (2013)** yaklaşımında, ardışık veri noktaları arasındaki bu aralıklar eğrinin her bir alt bölgesindeki esnekliği ve bükülme oranını belirleyen temel yapı taşlarıdır. Algoritma, hesaplanan bu fiziksel aralıkları bir sonraki adımda kurulacak olan denklem sisteminin katsayılarını oluşturmak üzere belleğe alır.

YÖNTEM

2. Üç Köşegenli Matrisin (Tridiagonal Matrix) Kurulması

Veri aralıkları hesaplandıktan sonra, aranan bükülme katsayılarını (k) bulmak için sistemin matematiksel iskeletini oluşturan denklem matrisi kurulur. Kübik spline algoritmasının doğası gereği, eğri üzerindeki her bir iç nokta yalnızca kendi solundaki ve sağındaki komşu noktalarla etkileşime girer. Bu nedenle, denklem sisteminin katsayıları standart ve yoğun bir matris yerine, yalnızca ana köşegeni ve onun birer alt/üst komşu köşegenlerini barındıran son derece sade bir Üç Köşegenli Matris (Tridiagonal Matrix) formu alır.

Kiusalaas (2013) formülasyonuna göre, sistemin rastgele bir i. iç noktasındaki türev sürekliliği (eğim) şartından türetilen matris katsayıları şu formüllerle tanımlanır:

- Alt Köşegen Katsayısı (c): Sol komşu nokta ile olan bağıntı ($x_{(i-1)} - x_i$).
- Ana Köşegen Katsayısı (d): Noktanın her iki komşusuyla olan toplam aralık bağıntısının iki katı: $2(x_{(i-1)} - x_{(i+1)})$.
- Üst Köşegen Katsayısı (e): Sağ komşu nokta ile olan bağıntı ($x_i - x_{(i+1)}$).
- Sağ Taraf Sonuç Vektörü (b): Komşu noktalar arasındaki eğimlerin farkının 6 katı olarak hesaplanan ve o satırdaki denklemin bilinen sonucunu oluşturan sağ taraf sabiti:

$$6\left[\frac{y_{i-1} - y_i}{x_{i-1} - x_i} - \frac{y_i - y_{i+1}}{x_i - x_{i+1}}\right] \quad (1)$$

3. Sınır Koşullarının (Boundary Conditions) Matematiksel Analizi

Kübik Spline interpolasyonunda, n+1 adet veri noktası (x_0, x_1, ..., x_n) için sistemin süreklilik şartları yalnızca iç düğümlerde (i = 1, 2, ..., n-1) tanımlanabilmektedir. Bu durum, n+1 adet bilinmeyen bükülme katsayısına (k_0, k_1, ..., k_n) karşılık yalnızca n-1 adet denklem üretir. **Kiusalaas (2013)** tarafından sunulan sayısal formülasyona göre, lineer denklem sisteminin (Üç Köşegenli Matrisin) tam ve benzersiz (unique) bir çözüme ulaşabilmesi için eğrinin x_0 ve x_n uç noktalarındaki fiziksel davranışını dikkate alan iki adet ek sınır koşulu denklemine ihtiyaç vardır.

Problemin fiziksel veya aerodinamik gereksinimlerine bağlı olarak, algoritmanın uç noktalarındaki davranışı iki temel sınır koşulu ile modellenir:

3.1. Doğal Sınır Koşulu (Natural Spline):

Eğrinin başlangıç ve bitiş noktalarında herhangi bir dış momentin veya bükülme kuvvetinin olmadığı serbest uç senaryolarında uygulanır. Fiziksel bir analojiyle, elastik cetvelin uçlarının serbest bırakılması durumudur.

- Matematiksel Tanım: Uç noktalardaki ikinci türevler (bükülme oranları) doğrudan sıfıra eşitlenir.
- Sistem Denklemleri: $k_0 = 0$ ve $k_n = 0$

3.2. Kenetlenmiş Sınır Koşulu (Clamped / Specified First Derivative)

Kübik spline eğrisinin başlangıç veya bitiş açısının (türevinin) önceden bilindiği veya fiziksel bir zorunluluk gereği belirli bir değere sabitlenmesi gerektiği durumlarda uygulanan bir yöntemdir. Örneğin, havacılıkta bir uçağın Sürükleme Kutbu (Drag Polar) grafiği çizilirken, minimum sürükleme katsayısının ($C_{(D(min))}$) bulunduğu tepe noktasında eğimin matematiksel olarak sıfır ($f'(x) = 0$) olması fiziksel bir kuraldır. Algoritmanın bu kurala uyması için sınır koşulu "kenetlenmiş" (clamped) olarak ayarlanır.

YÖNTEM

- Matematiksel Tanımı: Eğrinin uç noktalarının serbest bırakılması yerine, bu noktalardaki birinci türevlerin (teğet eğimlerinin) önceden belirlenmiş sabit bir değere eşitlenmesi işlemidir.

- Sisteme Etkisi: Başlangıç noktası olan x_0 'daki eğim ($f'(x_0)$) bilindiğinde, Kübik Spline'ın birinci türev denklemi bu özel nokta için hesaplanır. **Kiusalaas (2013)** formülasyonuna göre bu işlem, k_0 ve k_1 bükülme katsayıları arasında yeni bir doğrusal denklem ortaya çıkarır. Sürükleme Kutbu grafiğindeki gibi tepe noktasında eğimin sıfır ($f'(x_0) = 0$) olduğu varsayıldığında, sistemin ilk denklemi şu şekli alır:

$$2k_0 + k_1 = 6\,x\,\frac{y_1 - y_0}{(x_1 - x_0)^2} \qquad \textbf{(2)}$$

- Matrise Etkisi: Kenetlenmiş sınır koşulu, çözüm için kullanılan Üç Köşegenli Matrisin (Tridiagonal Matrix) genel yapısını bozmaz. Yalnızca matrisin 1. satırında (veya uygulandığı son satırda) bulunan ana köşegen ve üst köşegen değerlerini günceller. Bu müdahale sayesinde algoritma; eğrinin iç noktalarındaki katsayıları hesaplarken, sistemin başlangıç açısını kullanıcının veya fiziksel kuralın belirlediği o zorunlu eğime sabitler ve tüm sistemi bu şarta bağlı kalarak çözer.

4. Üç Köşegenli Matrislerin LU Ayrıştırması ve Thomas Algoritması

Kübik Spline interpolasyonunun temelini oluşturan eşzamanlı lineer denklem sistemleri, düğüm noktalarındaki birinci ve ikinci türevin birbirine eşit olması şartı nedeniyle süreklilik koşulunu sağlamasından dolayı Üç Köşegenli Matris (Tridiagonal Matrix) yapısındadır. Bu özel matrislerin sayısal çözümü için **Kiusalaas (2013)** tarafından önerilen optimize edilmiş Thomas Algoritması, matrisi parçalamaya dayalı LU Ayrıştırması (LU Decomposition) yöntemi ile uygulanmaktadır.

Geleneksel matris gösterimlerinde N boyutlu bir denklem sistemi N x N boyutlarında iki boyutlu bir dizi gerektirirken; bu algoritmada matrisin sıfırlardan oluşan kısımları belleğe alınmaz. Bunun yerine matris, yalnızca dolu olan üç köşegeni temsil eden tek boyutlu üç adet vektör ile tanımlanır:

$$\begin{bmatrix} d_1 & e_1 & 0 & \cdots \\ c_1 & d_2 & e_2 & \cdots \\ 0 & c_2 & d_3 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \end{bmatrix} \qquad \textbf{(3)}$$

- c vektörü: Alt köşegen elemanları (ilk değişken)
- d vektörü: Ana köşegen elemanları (ikinci değişken)
- e vektörü: Üst köşegen elemanları (üçüncü değişken)
- b vektörü: Eşitliğin sağ tarafındaki bilinen sonuçlar(3. Dereceden denklem sonuçları)
- Sistemin çözüm süreci, bellek (RAM) kullanımını minimum düzeye indirmek amacıyla "yerinde değiştirme" (in-place modification) prensibiyle iki ana aşamada gerçekleştirilir:

YÖNTEM

4.1. LU Ayrıştırma Aşaması (Decomposition Phase)

Bu aşamanın amacı, katsayılar matrisini Alt (Lower) ve Üst (Upper) üçgensel matrislere ayırmaktır. Sistem baştan sona tek bir döngü içerisinde taranır ve k = 1, 2, ..., n-1 indeksleri için aşağıdaki işlemler sırasıyla uygulanır:

4.1.a. İlk adımdaki temel amacımız, matrisin alt köşegenini (Yani c vektörünü) matematiksel olarak yok etmek, yani sıfırlamaktır. Bunu başarabilmek için, üst satırın alt satırı hangi oranda yok edeceğini hesaplamamız gerekir; c ile d değerleri arasındaki bu sayısal orana 'çarpan' denir.

Bilgisayar hesapladığı bu çarpanı sistem belleğinde (RAM) yeni bir değişken açıp boşuna yer işgal etmek yerine, çok optimize bir yaklaşımla, zaten yok ettiğimiz o, c vektörünün boşalan yerine doğrudan kaydeder (üzerine yazar): $c_{(k-1)} = c_{(k-1)} / d_{(k-1)}$

$$\begin{bmatrix} d_1 & e_1 & 0 & 0 & \cdots & 0 \\ c_1 & d_2 & e_2 & 0 & \cdots & 0 \\ 0 & c_2 & d_3 & e_3 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & c_{n-2} & d_{n-1} & e_{n-1} \\ 0 & 0 & \cdots & 0 & c_{n-1} & d_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \\ b_n \end{bmatrix} \quad \textit{(4)}$$

4.1.b. Alt köşegeni matematiksel olarak yok etmek için üst satırı kendi satırımızdan çıkardığımızda, üst satırda bulunan 'üst köşegen' (e vektörü) elemanları mecburen denklemimize etki eder. Eşitliğin bozulmaması ve sistemin dengede kalması için ana iskeletimiz olan 'ana köşegeni' (d vektörü) bu yeni duruma göre sağlamlaştırmamız (güncellememiz) şarttır.

Bu güncelleme hesabı yapılırken, az önce bulup c'nin içine kaydettiğimiz o 'çarpan' ile üstten gelen bu mecburi e etkisi kullanılır. Bilgisayar, elde ettiği bu yeni ve güncel d değerini yine bellekte (RAM) yeni bir liste açıp yer israf etmek yerine, son derece optimize bir şekilde doğrudan eski d vektörünün üzerine yazar. ($d_k = d_k - c_{(k-1)} * e_{(k-1)}$)

Bu aşamanın sonunda matrisin alt kısmı tamamen sıfırlanmış ve ana köşegeni (d vektörü) yeni değerlerle güncellenmiş olur.

$$\begin{bmatrix} d'_1 & e_1 & 0 & 0 & \cdots & 0 \\ 0 & d'_2 & e_2 & 0 & \cdots & 0 \\ 0 & 0 & d'_3 & e_3 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & d'_{n-1} & e_{n-1} \\ 0 & 0 & \cdots & 0 & 0 & d'_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \\ b_n \end{bmatrix} \quad \textit{(5)}$$

YÖNTEM

4.2. Sayısal Çözüm Aşaması (Solution Phase)

Ayrıştırılmış matris üzerinden bilinmeyen katsayıların (Kübik spline bükülme değerlerinin) elde edilmesi işlemi iki adımda tamamlanır. Bu işlemde de ekstra bir çözüm vektörü oluşturulmaz; bulunan tüm sonuçlar eşitliğin sağ tarafındaki b vektörünün üzerine yazılır.

4.2.a. İleri Yönlü Yerine Koyma (Forward Substitution): Ayrıştırma aşamasında denklemin sol tarafındaki katsayıları değiştirip sistem sadeleştiriildiği için, matematiğin temel eşitlik kuralı gereği sistemin dengesi bozulur. Eşitliği yeniden sağlamak amacıyla, sol tarafa uygulanan 'çıkarma' işlemlerinin birebir aynısını eşitliğin sağ tarafındaki sonuç vektörüne (b) de uygulanması zorunludur.

Bu dengeleme işlemi için en üst satırdan başlanarak aşağıya doğru inilir. Her bir satırın yeni sonucunu hesaplarken; ilk aşamada bulunup c vektörü yerine yazılan 'çarpan' ile, bir üst satırın sonucu ($b_{(k-1)}$) çarpılır ve istenilen satır sonucundan (b_k) çıkarılır. Bilgisayar, sistemi dengeleyen bu yeni sonucu da yine bellekte (RAM) yeni bir liste açıp yer işgal etmek yerine, doğrudan b vektörünün eski değerinin üzerine yazar. $b_k = b_k - c_{(k-1)} * b_{(k-1)}$

4.2.b. Geriye Doğru Yerine Koyma (Backward Substitution): LU ayrıştırma aşamasında sistemin katsayılar matrisine uygulanan satır operasyonları, matematiksel eşitliğin korunması zorunluluğu gereği eşitliğin sağ tarafındaki sonuç vektörüne (b) de aynı şekilde uygulanmalıdır. İleri yönlü yerine koyma (forward substitution) adı verilen bu dengeleme sürecinde, işlemler sistemin ilk satırından başlanarak aşağıya doğru sırasıyla gerçekleştirilir.

Sağ taraf vektörünün her bir elemanını güncellemek için; önceki aşamada hesaplanarak alt köşegen (c) vektörüne kaydedilen çarpan (yok etme katsayısı), bir önceki satırın güncellenmiş sonucu ($b_{(k-1)}$) ile çarpılarak mevcut satırın değerinden (b_k) çıkarılır. Kiusalaas (2013) algoritmasının bellek optimizasyonu prensibine sadık kalınarak, hesaplanan bu yeni değerler için sistem belleğinde ilave bir çözüm dizisi tahsis edilmez;

Elde edilen sonuçlar doğrudan mevcut b vektörünün üzerine (in-place) yazılarak algoritmanın donanımsal verimliliği korunur: ($b_{son} = b_{son} / d_{son}$)

Ardından, aşağıdan yukarıya doğru (k = n-2, n-3, ..., 0) dönülerek tüm bilinmeyen katsayılar ardışık hesaplamalarla bulunur: ($b_k = (b_k - e_k * b_{(k+1)}) / d_k$)

5. Veri Konumlandırma ve İkiye Bölme (Bisection) Algoritmasının Matematiği

Katsayılar matrisinin çözülüp her bir aralığa ait kübik polinom bükülme katsayılarının (k_i) belleğe alınmasının ardından, interpolasyon işleminin tamamlanması için sistemden talep edilen rastgele bir ara değerin (x_{hedef}) veri seti içindeki konumunun tespit edilmesi gerekmektedir. Gerçek zamanlı sistemlerde, bağımsız değişkenler ($x_0, x_1, ..., x_n$) doğası gereği küçükten büyüğe sıralı bir dizi oluşturur. İkiye Bölme (Bisection) algoritması, bu sıralı veri yapısını kendi avantajına kullanarak "Parçala ve Yönet" mantığıyla çalışır.

YÖNTEM

Geleneksel bir Doğrusal Arama (Linear Search) algoritması, aranan hedef değerin hangi iki düğüm (x_i ve $x_{(i+1)}$) arasına düştüğünü bulmak için diziyi ilk elemandan itibaren sırayla kontrol eder. Ancak bu yaklaşım, algoritmanın mantıksal anatomisinde İkiye Bölme yöntemi ile şu şekilde optimize edilmiştir:

Bisection algoritmasının temel işlevi, sıralı bir veri seti (örneğin aerodinamik katsayılar dizisi) içerisinde aranan hedef değerin hangi iki veri noktası arasına düştüğünü bulmaktır. Veri setini baştan sona tek tek tarayan standart yöntemlerin aksine, bu algoritma arama uzayını her iterasyonda tam ortadan ikiye bölerek çok daha hızlı ve verimli bir sonuç üretir. Algoritma şu adımları izleyerek çalışır:

5.1. Arama Uzayının Tanımlanması:

Sistem belleğinde taranacak olan dizinin en başı alt sınır (L = 0), en sonu ise üst sınır (R = n) olarak atanır. Arama işlemi, bu iki indeksin aralarındaki mesafe 1'den büyük olduğu sürece (R - L > 1) devam edecek bir döngü başlatır.

5.2. Orta Noktanın Hesaplanması:

Döngünün her adımında, mevcut alt ve üst sınırların toplanıp ikiye bölünmesiyle arama alanının tam orta indeksi (M) hesaplanır:

$$M = \left\lfloor \frac{L + R}{2} \right\rfloor \quad \textit{(6)}$$

(Not: İndekslerin her zaman tam sayı olması gerektiğinden, bu işlemde tam sayı bölmesi kullanılır).

5.3. Hedefin Karşılaştırılması ve Alanın Daraltılması:

Sistemin aradığı hedef değer ($x_{(hedef)}$), dizinin tam ortasına denk gelen değer (x_M) ile kıyaslanır. Bu kıyaslama sonucunda arama uzayı yarı yarıya daraltılır:

- Eğer $x_{(hedef)} < x_M$ ise; aranan hedefin dizinin sol yarısında olduğu kesinleşir. Bu durumda sağ taraftaki tüm veriler gözden çıkarılarak, üst sınır doğrudan orta noktaya çekilir (R = M).

- Eğer $x_{(hedef)}$ \ge x_M ise; aranan hedefin dizinin sağ yarısında olduğu kesinleşir. Bu durumda sol taraftaki veriler gözden çıkarılır ve alt sınır orta noktaya kaydırılır (L = M).

5.4. Döngünün Sonlanması ve Kıstırma Şartı:

Sınırların sürekli daraltıldığı bu ikiye bölme işlemi, L ve R indeksleri ardışık konuma gelene kadar sürer. İki indeks arasındaki fark 1'e düştüğünde (R - L = 1), algoritma aranan hedef değerin x_L ile x_R noktaları arasına başarıyla kıstırıldığını matematiksel olarak ispatlamış olur. Döngü sonlandırılır ve algoritma, hedefin bulunduğu aralığın başlangıç noktası olan L indeksini ana sisteme geri döndürür.

YÖNTEM

6. Ara Değerin (İnterpolasyon Sonucunun) Hesaplanması

Kübik Spline algoritmasının nihai aşaması, bağımsız değişkenin (sistemden talep edilen hedef x değerinin) tanımlı aralık içerisindeki karşılığının (y(x) ara değerinin) matematiksel olarak elde edilmesidir. Bu aşamaya geçilmeden önce; İkiye Bölme (Bisection) algoritması ile hedef noktanın x_i ve $x_{(i+1)}$ düğüm noktaları (knots) arasında yer aldığı tespit edilmiş ve Thomas Algoritması (LU Ayrıştırması) ile bu noktalara ait ikinci türev (bükülme) katsayıları (k_i, $k_{(i+1)}$) çözümlenmiş olmalıdır.

Kiusalaas (2013) metodolojisine göre, tespit edilen alt aralık ([x_i, $x_{(i+1)}$]) içerisinde fonksiyonun davranışını tanımlayan üçüncü dereceden (kübik) interpolasyon polinomu, aralığın sınır koordinatları ve hesaplanan bükülme katsayıları kullanılarak şu şekilde ifade edilir:

$$y(x) = \frac{k_i}{6}\left[\frac{(x_{i+1}-x)^3}{x_{i+1}-x_i} - (x_{i+1}-x)(x_{i+1})\right] + \frac{k_{i+1}}{6}\left[\frac{(x-x_i)^3}{x_{i+1}-x_i} - (x-x_i)(x_{i+1})\right] \quad \textbf{(7)}$$

Matematiksel Yapı ve Fiziksel Anlamı:

Bu interpolasyon polinomu, yapısal olarak iki temel matematiksel bileşenin süperpozisyonundan (üst üste binmesinden) oluşmaktadır. Formülün son iki terimi, x_i ve $x_{(i+1)}$ sınır noktaları arasında kurulan basit bir doğrusal interpolasyonu (linear ninterpolation) temsil eder. Eşitliğin başındaki k_i ve $k_{(i+1)}$ katsayılarını içeren kübik terimler ise; bu doğrusal iskelete, komşu veri noktalarıyla birinci ve ikinci türev sürekliliğini sağlayacak olan eğrilik (bükülme) fonksiyonunu ilave eder.

Sonuç olarak hesaplanan bu y(x) ara değeri, veri seti (örneğin aerodinamik sürükleme profili) üzerinde C^2 sınıfı (ikinci türeve kadar sürekli) bir pürüzsüzlük sağlar. Algoritma, düğüm noktaları arasında hiçbir keskin köşe veya matematiksel süreksizlik barındırmayan, fiziksel gerçekliğe en yakın değeri üreterek interpolasyon sürecini başarıyla tamamlar.

Sonuç ve Sürecin Özeti

Kısaca özetlemek gerekirse, kurulan bu interpolasyon mimarisi baştan sona şu mantıkla işlemektedir: İlk olarak elimizdeki aerodinamik verilerin aralıkları hesaplanıp sistemin matris iskeleti kurulmuştur. Ardından, uçağın fiziksel kurallarına (minimum sürükleme noktasındaki sıfır eğim kuralına) uyması için kenetlenmiş sınır koşulları bu matrise entegre edilmiştir. Kurulan bu denklem sistemi Thomas Algoritması ile bellek israfı yapılmadan çözülmüş ve eğrinin tüm bükülme katsayıları (k) bulunmuştur. Bükülme katsayıları elde edildikten sonra, İkiye Bölme (Bisection) yöntemiyle talep edilen hedef değerin hangi iki veri noktası arasına düştüğü hızla tespit edilmiştir. Son aşamada ise, tespit edilen bu aralık ve hesaplanan bükülme katsayıları kübik interpolasyon polinomuna yerleştirilerek, grafikteki o nihai ve pürüzsüz ara değer hesaplanmış ve süreç başarıyla tamamlanmıştır.

Referanslar :

1) Kiusalaas, J. (2013). Numerical Methods in Engineering with Python 3. Cambridge University Press.